

TRƯỜNG ĐẠI HỌC TÔN ĐỨC THẮNG
KHOA CÔNG NGHỆ THÔNG TIN

BÁO CÁO CHỨC NĂNG GIỮA KỲ
MÔN: KIẾN TRÚC HƯỚNG DỊCH VỤ (SOA)

NOTIFICATION SERVICE – THÀNH VIÊN 6

THÔNG TIN	NỘI DUNG
HỌ VÀ TÊN	Nguyễn Đức Nhật Huy
MSSV	52400122
VAI TRÒ	Thành viên 6 – phát triển Notification Service bằng Go
NGÔN NGỮ	Go
HTTP	Cổng 5006
DATABASE	MySQL – notification_db
TRIỂN KHAI CSDL	Docker Compose

Báo cáo tập trung vào các sản phẩm P1–P8 của Notification Service.

MỤC LỤC

1. GIỚI THIỆU VÀ PHẠM VI.....	3
1.1. Phạm vi kỹ thuật	3
1.2. Phạm vi chưa triển khai ở giữa kỳ.....	3
2. P1 – CƠ SỞ DỮ LIỆU + SEED	3
2.1. Bảng notification_logs	3
2.2. Schema và seed.....	4
3. P2 – ERD	4
4. P3 – USE CASE.....	4
5. P4 – DFD MỨC 0 VÀ MỨC 1	5
5.1. DFD mức 0	5
5.2. DFD mức 1	6
6. P5 – DFD MỨC 2	6
7. P6 – API, MÃ NGUỒN VÀ KIỂM THỬ.....	6
7.1. API.....	6
7.2. Kiểm thử mã nguồn	6
7.3. Giới hạn kiểm thử cần ghi rõ	7
8. P7 – TRANG ADMIN	8
9. P8 – TỔNG HỢP, HƯỚNG DẪN CHẠY VÀ TỰ KIỂM TRA.....	9
9.1. Chạy MySQL bằng Docker	9
9.2. Cấu hình Notification Service	9
9.3. Chạy trang Admin	9
9.4. Checklist P1–P8.....	9
10. KẾT LUẬN VÀ ĐỊNH HƯỚNG CUỐI KỲ.....	10
10.1. Các nội dung đã hoàn thành	10
10.2. Định hướng cuối kỳ	10

1. GIỚI THIỆU VÀ PHẠM VI

Notification Service là service chuyên xử lý việc gửi thông báo xác nhận và lưu nhật ký thông báo trong hệ thống đặt vé xem phim theo kiến trúc microservices. Service nhận yêu cầu từ Booking Service, tự tạo nội dung, thực hiện gửi giả lập và ghi log vào CSDL riêng.

Phiên bản project hiện tại đã chuyển phần CSDL sang Docker Compose để việc khởi tạo MySQL thuận tiện và nhất quán hơn. Ứng dụng Go vẫn kết nối MySQL thông qua các biến môi trường.

Trong phạm vi giữa kỳ, service tập trung vào ba nghiệp vụ: gửi thông báo xác nhận, ghi nhật ký thông báo và xem nhật ký thông báo.

1.1. Phạm vi kỹ thuật

- Backend viết bằng Go, sử dụng net/http, encoding/json và database/sql.
- MySQL 8.0, database notification_db và bảng notification_logs.
- REST API gồm POST /api/notify và GET /api/notify/logs.
- Trang admin chỉ xem nhật ký, có tải lại và bộ lọc.
- Docker Compose dùng để chạy MySQL; service không chạy fake mode khi DB không kết nối.

1.2. Phạm vi chưa triển khai ở giữa kỳ

- Email/SMS thật.
- JWT hoặc X-Internal-Key.
- Queue/outbox, retry/idempotency.
- Tích hợp hoàn chỉnh với các service khác trong môi trường production.

2. P1 – CƠ SỞ DỮ LIỆU + SEED

Notification Service áp dụng nguyên tắc database-per-service. Service chỉ sở hữu và thao tác trên notification_db; booking_id là tham chiếu logic đến Booking Service, không tạo foreign key vật lý giữa hai database.

2.1. Bảng notification_logs

Cột	Kiểu dữ liệu	Ràng buộc / ý nghĩa
id	INT	PRIMARY KEY, AUTO_INCREMENT
booking_id	INT	NOT NULL; mã booking
username	VARCHAR(50)	NOT NULL; người nhận
channel	VARCHAR(20)	NOT NULL; EMAIL hoặc SMS
message	VARCHAR(255)	NOT NULL; nội dung thông báo
sent_at	DATETIME	NOT NULL; thời điểm gửi UTC

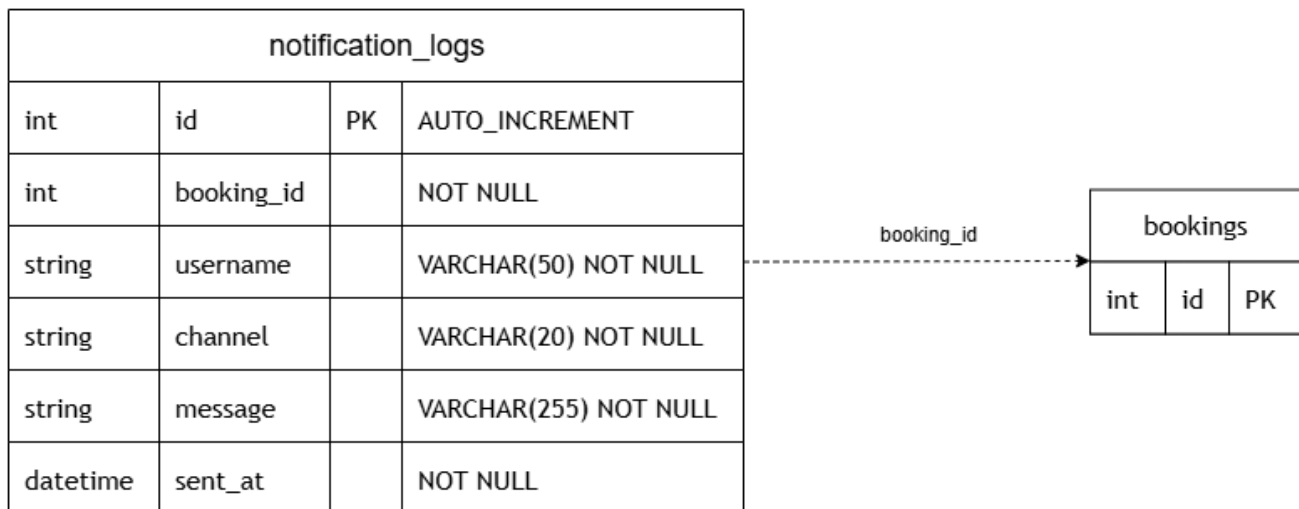
Project có index idx_sent_at_id trên (sent_at, id) để hỗ trợ truy vấn nhật ký mới nhất.

2.2. Schema và seed

- db/schema.sql dùng CREATE DATABASE IF NOT EXISTS và CREATE TABLE IF NOT EXISTS.
- db/seed.sql tạo dữ liệu mẫu cho booking_id=1, username=user1, channel=EMAIL và nội dung xác nhận.
- Seed dùng GET_LOCK và WHERE NOT EXISTS để hạn chế tạo trùng khi chạy lặp.
- Không sử dụng DROP/TRUNCATE để xóa dữ liệu hiện có.
- Docker Compose cung cấp MySQL 8.0 và volume mysql_data.

3. P2 – ERD

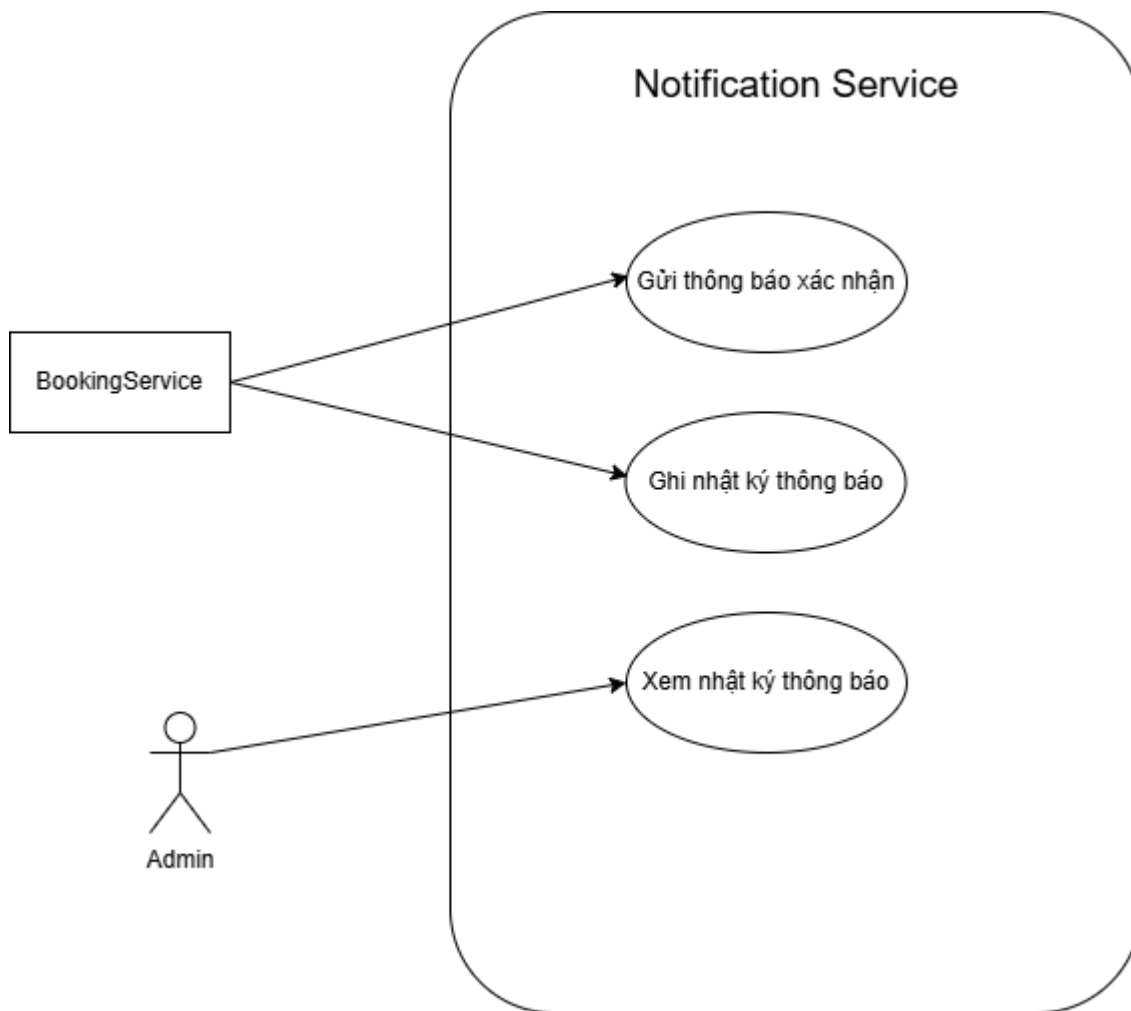
ERD mô tả bảng notification_logs là kho dữ liệu thuộc Notification Service. booking_id thể hiện quan hệ nghiệp vụ logic với Booking Service.



HÌNH P2 – ERD NOTIFICATION SERVICE

4. P3 – USE CASE

Actor gồm Booking Service và Quản trị viên. Các use case chính được thống nhất cho Notification Service là: Gửi thông báo xác nhận, Ghi nhật ký thông báo và Xem nhật ký thông báo.



HÌNH P3 – USE CASE

Actor	Use case	Mô tả
Booking Service	Gửi thông báo xác nhận	Gửi username, bookingId và channel
Notification Service	Ghi nhật ký thông báo	Lưu dữ liệu gửi vào notification_logs
Quản trị viên	Xem nhật ký thông báo	Tra cứu nhật ký thông qua API/trang admin

5. P4 – DFD MỨC 0 VÀ MỨC 1

5.1. DFD mức 0

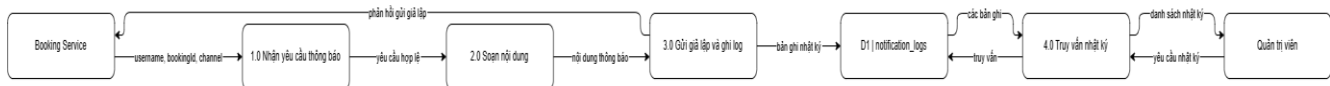
Ở mức ngữ cảnh, Booking Service gửi yêu cầu thông báo đến Notification Service. Quản trị viên gửi yêu cầu xem nhật ký và nhận dữ liệu nhật ký.



HÌNH P4.1 – DFD MỨC 0

5.2. DFD mức 1

Mức 1 phân rã Notification Service thành: 1.0 Nhận yêu cầu thông báo; 2.0 Soạn nội dung; 3.0 Gửi giả lập và ghi log; 4.0 Truy vấn nhật ký. Kho dữ liệu D1 là notification_logs.



HÌNH P4.2 – DFD MỨC 1

6. P5 – DFD MỨC 2

Tiến trình 3.0 được phân rã thành 3.1 Chọn kênh gửi; 3.2 In/gửi giả lập; 3.3 Ghi log; 3.4 Tạo phản hồi. Luồng vào và luồng ra phải cân bằng với tiến trình 3.0 của DFD mức 1.



HÌNH P5 – DFD MỨC 2

7. P6 – API, MÃ NGUỒN VÀ KIỂM THỬ

7.1. API

Method	Endpoint	Chức năng
POST	/api/notify	Nhận username, bookingId, channel; tạo nội dung; gửi giả lập; ghi log
GET	/api/notify/logs	Truy vấn nhật ký từ MySQL
OPTIONS	/api/notify	CORS preflight
OPTIONS	/api/notify/logs	CORS preflight

POST /api/notify sử dụng JSON gồm username, bookingId và channel. Service tự tạo message. Kênh hợp lệ là EMAIL hoặc SMS. Response thành công và lỗi dùng envelope thống nhất.

Service giới hạn request body 16 KiB, kiểm tra JSON không hợp lệ, trailing JSON, username, bookingId và channel; xử lý các trường hợp 404, 405, 400, 413 và 500.

7.2. Kiểm thử mã nguồn

Kiểm tra	Kết quả đã ghi nhận
go mod tidy	PASS
gofmt	PASS
go vet ./...	PASS

go test ./... -count=1 -cover	PASS – coverage 57.0%
go build ./...	PASS
Build binary notification.exe	PASS
node --check script admin	PASS
Python HTTP server admin	HTTP 200 cho các trang đã kiểm tra

7.3. Giới hạn kiểm thử cần ghi rõ

- Unit test sử dụng fake store/sender nên không thay thế integration test với MySQL thật.
- Môi trường xác minh trước đó chưa có Docker Engine/MySQL listener hoạt động, vì vậy cần chạy lại integration test trên máy demo.
- Cần chụp ảnh POST thành công, GET logs và dữ liệu notification_logs sau khi Docker/MySQL chạy thực tế.

```
PS D:\workspace\SOA\project_final> $body = @{ username = 'user1'; bookingId = 10; channel = 'EMAIL' } | ConvertTo-Json
>> $result = Invoke-RestMethod -Method Post -Uri 'http://localhost:5006/api/notify' -ContentType 'application/json; charset=utf-8' -Body ([System.Text.Encoding]::UTF8.GetBytes($body))
>> $result | ConvertTo-Json -Depth 6
{
  "status": "success",
  "data": {
    "notificationId": 3,
    "bookingId": 10,
    "username": "user1",
    "channel": "EMAIL",
    "deliveryStatus": "SIMULATED",
    "message": "Đặt vé thành công cho đơn #10",
    "sentAt": "2026-10-05T06:58:54Z"
  }
}
```

HÌNH P6.1 – POST thành công

```
PS D:\workspace\SOA\project_final> Invoke-RestMethod 'http://localhost:5006/api/notify/logs' | ConvertTo-Json -Depth 6
{
  "status": "success",
  "data": [
    {
      "id": 3,
      "bookingId": 10,
      "username": "user1",
      "channel": "EMAIL",
      "message": "Đặt vé thành công cho đơn #10",
      "sentAt": "2026-10-05T06:58:54Z"
    },
    {
      "id": 2,
      "bookingId": 3,
      "username": "user1",
      "channel": "EMAIL",
      "message": "Đặt vé thành công cho đơn #3",
      "sentAt": "2026-10-04T15:35:15Z"
    },
    {
      "id": 1,
      "bookingId": 1,
      "username": "user1",
      "channel": "EMAIL",
      "message": "Đặt vé thành công cho đơn #1",
      "sentAt": "2026-01-01T00:00:00Z"
    }
  ]
}
```

HÌNH P6.2 – GET thành công

	123 id	123 booking_id	AZ username	AZ channel	AZ message	🕒 sent_at
1	1	1	user1	EMAIL	Đặt vé thành công cho đơn #1	2026-01-01 00:00:00
2	2	3	user1	EMAIL	Đặt vé thành công cho đơn #3	2026-10-04 15:35:15
3	3	10	user1	EMAIL	Đặt vé thành công cho đơn #10	2026-10-05 06:58:54

HÌNH P6.3 – Dữ liệu thực tế trong database

8. P7 – TRANG ADMIN

Trang admin_notification.html là trang chỉ xem nhật ký Notification. Trang gọi GET /api/notify/logs, hiển thị bảng nhật ký và hỗ trợ lọc username, bookingId, channel.

Thành phần	Mô tả
Bảng nhật ký	Booking, người nhận, kênh, nội dung, thời gian
Tải lại	Gọi lại API để lấy dữ liệu mới
Bộ lọc	username / bookingId / channel
Trạng thái	loading / empty / filter / error
Hiển thị dữ liệu	Dùng textContent để tránh chèn HTML từ dữ liệu API

admin/index.html là khung 6 tab. Tab Notification được tích hợp; các tab của service khác chỉ nên đánh dấu đã tích hợp khi nhóm thực sự ghép được các trang tương ứng.

Nhật ký thông báo

Theo dõi thông báo đặt vé đã gửi giả lập qua EMAIL và SMS.

Người nhận hoặc mã đơn

Ví dụ: user1 hoặc 3

Kênh thông báo

Tất cả kênh

Hiển thị 2 / 2 nhật ký.

ID nhật ký	Mã đơn	Người nhận	Kênh	Nội dung	Thời gian
2	3	user1	EMAIL	Đặt vé thành công cho đơn #3	22:35:15 4/10/2026
1	1	user1	EMAIL	Đặt vé thành công cho đơn #1	07:00:00 1/1/2026

Nguồn API: http://localhost:5006 · Chỉ xem · Thời gian hiển thị theo múi giờ trình duyệt

P4	DFD 0 và 1	☒
P5	DFD 2	☒
P6	API + kiểm thử + ảnh	Code/test/build ☒ ; ảnh integration cần bổ sung
P7	Trang admin	Code ☒ ; ảnh chụp cần bổ sung
P8	Báo cáo	☒

10. KẾT LUẬN VÀ ĐỊNH HƯỚNG CUỐI KỲ

Notification Service đã hoàn thành phần giữa kỳ theo các sản phẩm P1–P8. Phiên bản hiện tại đã cập nhật Docker Compose cho MySQL, schema/seed riêng cho service, API REST/JSON, unit test, build và trang admin.

10.1. Các nội dung đã hoàn thành

- Thiết kế notification_logs và seed.
- ERD, Use Case, DFD mức 0/1 và DFD mức 2.
- POST /api/notify và GET /api/notify/logs.
- Validation, CORS/OPTIONS và xử lý lỗi.
- Unit test, go vet, gofmt và build.
- Trang admin Notification và khung admin chung.

10.2. Định hướng cuối kỳ

- X-Internal-Key/JWT ADMIN nếu được yêu cầu trong tích hợp cuối kỳ.
- Tích hợp luồng Booking → Notification thực tế.
- Email/SMS hoặc provider adapter.
- Retry/idempotency và queue/outbox nếu nhóm áp dụng.
- Integration test với Docker/MySQL và kiểm tra đầy đủ giao diện admin.

— HẾT —